Разработка модулей ядра Linux: Часть 2. Анализ полученных результатов

Олег Цилюрик
преподаватель тренингового отделения
Global Logic

26.04.2012

Эта статья продолжает цикл статей, посвященный написанию модулей ядра Linux. В статье представлен краткий обзор структуры модуля ядра.

Введение

В предыдущей статье данного цикла были разработаны три модуля ядра Linux и продемонстрированы их возможности, однако при этом не было представлено никакой информации, объясняющей, что конкретно было сделано, и как этим пользоваться. Ответы на эти и другие вопросы будут даны в данной статье.

Структура модуля

В предыдущей части был создан первый ознакомительный проект (архив export-data.tgz), в который вошли три зависимых модуля ядра. Теперь можно пристально изучить то, что получилось в итоге, и извлечь из этого учебного проекта уроки, относящиеся к модульному программированию. На платформе Linux есть псевдофайл /proc/kallsyms, содержащий **динамическую** таблицу всех символов (имена функций, структур данных и т.д.), известных в ядре. Вот как выглядит в этой таблице рассматривавшийся ранее вызов функции `printk()`.

Листинг 1. Фрагмент содержимого таблицы символов

```
$ cat /proc/kallsyms | grep T | grep ' printk'
c040bcd1 T printk_address
c044136b T printk_needs_cpu
c04415d8 T printk_timed_ratelimit
c0441617 T printk_ratelimit
c0441816 T printk_tick
c07a3955 T printk
c0a18664 T printk_all_partitions
```

Стоит отметить, что `c07a3955` — это абсолютный адрес точки входа в адресном пространстве ядра; эта информация будет использоваться далее в статье. Далее следует произвести повторную загрузку модулей, созданных на предыдущем шаге:

© Copyright IBM Corporation 2012

```
$ sudo insmod md1.ko$ sudo insmod md2.ko$ lsmod | grep md
md2                     646  0
md1                     860  1 md2
```

После этого можно снова выполнить обращение к таблице символов, чтобы найти имена ядра, привнесенные новыми модулями.

Листинг 2. Символы ядра, порожденные новыми модулями

```
$ cat /proc/kallsyms | grep '\[md'
f8b19000 t md_exit [md2]
f8b190b8 d __this_module [md2]
f8b19000 t cleanup_module [md2]
f8ad7020 t md_exit [md1]
f8ad70a0 r __ksymtab_md1_proc [md1]
f8ad70b0 r __kstrtab_md1_proc [md1]
f8ad70a8 r __ksymtab_md1_data [md1]
f8ad70b9 r __kstrtab_md1_data [md1]
f8ad70cc D md1_data [md1]
f8ad70d0 d __this_module [md1]
f8ad7000 T md1_proc [md1]
f8ad7020 t cleanup_module [md1]
```

Как видно из листинга 2, **динамическая** таблица имён дополнилась именами загруженных модулей. Термин «динамическая» специально выделен, так как в системе существует и другая таблица имён ядра, формируемая на момент компиляции ядра и обычно хранящаяся в каталоге **/boot**.

Листинг 3. Статическая таблица символов ядра

```
$ cat /boot/System.map-`uname -r` | grep T | grep ' printk'
c040bcd1 T printk_address
c044136b T printk_needs_cpu
c04415d8 T printk_timed_ratelimit
c0441617 T printk_ratelimit
c0441816 T printk_tick
c07a3955 T printk
c0a18664 T printk_all_partitions
```

На эту таблицу часто по ошибке ссылаются в обсуждениях, так как эта таблица **статическая**, зафиксированная на момент её формирования, и, конечно, она не претерпевает никаких изменений в зависимости от подгружаемых позже компонентов ядра, как видно из представленного примера.

```
$ cat /boot/System.map-`uname -r` | grep T | grep '\[md'
$
```

Примечание. В представленных примерах в командной строке выполнена конкатенация с результатом другой команды:

$ uname -r 2.6.32.9-70.fc12.i686.PAE
Часто при выполнении подобных команд возникают ошибки, связанные с тем, что символ апострофа для придания ему такого смысла должен набираться с клавиши, соответствующей символу тильде: **~**, а не с других символов кавычек, имеющихся на клавиатуре.

Продолжим анализ **динамической** таблицы имён, приведенной в листинге 2. Характерно то, что в этой таблице дважды встречается имя `md_exit`: относящееся к модулю `md1` и к модулю `md2`, но отсутствует **симметричное** ему имя `md_init`, которое, казалось бы, тоже должно быть в таблице. Это связано с квалификатором `__init`, указанном в описании функции `md_init()`, который предписывает загрузчику модуля освободить память, занимаемую кодом этой функции, после её (функции) однократного выполнения (точнее, просто отметить её как границу свободной памяти, доступной для загрузки следующих модулей). Что представляет собой квалификатор `__init` с точки зрения синтаксиса? Он часто встречается при программировании ядра и является макроопределением препроцессорной функции без параметров. Также важен вопрос, почему имена `md_exit`, одинаково определённые в нескольких разных модулях, не конфликтовали между собой? Дело в том, что они специально были объявлены как локальные с использованием квалификатора `static`.

Ещё одним полезный инструмент для анализа имён модуля - это утилита **nm** для работы с объектными модулями gcc. Она не имеет никакого отношения к программированию ядра, но стоит напомнить, что скомпилированный модуль является и объектным модулем!

Листинг 4. Анализ символов модуля

```
$ nm md1.ko | grep ' md'
00000000 D md1_data
0000000f T md1_noexport
00000000 T md1_proc
00000000 t md_exit
00000000 t md_init
$ nm md2.ko | grep ' md'
         U md1_data
         U md1_proc
00000000 t md_exit
00000000 t md_init
```

Эта же утилита, а точнее её справочник man, содержит ключ для чтения таблиц символов ядра (динамической и статической), о которых говорилось ранее, и расшифровывает буквенный код, означающий символ имени (локальный или глобальный, точка входа функции, объект данных, и т.д.).

Листинг 5. Коды символов в объектном формате

```
$ man nm
NAME
       nm - list symbols from object files
...
       ·   The symbol type.  At least the following types are used; others are, as
           well, depending on the object file format.  If lowercase, the symbol is
           local; if uppercase, the symbol is global (external).
...
           "D"
           "d" The symbol is in the initialized data section.
...
           "T"
           "t" The symbol is in the text (code) section.
           "U" The symbol is undefined.
```

Из других инструментов при исследовании модулей полезными могут оказаться objdump и readelf. В статье они рассматриваться не будут, но эти инструменты выводят большой объем информации и обладают богатыми возможностями, поэтому с ними стоит познакомиться.

```
$ objdump -h md1.ko
md1.ko:     file format elf32-i386
Sections:
...
$ readelf -s md1.ko
Symbol table '.symtab' contains 45 entries:
```

Еще один вопрос, который стоит рассмотреть: чем формат модулей **.ko** отличается от объектного формата **.o** (на самом деле, файл в данном формате создаётся **make** на промежуточном этапе выполнения)? Для ответа него придется прибегнуть к помощи утилиты уже из комплекта для модульного программирования:

Листинг 6. Форматы объектных файлов

```
$ modinfo md2.o
filename:       md2.o
author:         Oleg Tsiliuric <olej@front.ru>
license:        GPL
$ modinfo md2.ko
filename:       md2.ko
author:         Oleg Tsiliuric <olej@front.ru>
license:        GPL
srcversion:     3F6EDACE7C8B9120F773156
depends:        md1
vermagic:       2.6.32.9-70.fc12.i686.PAE SMP mod_unload 686
```

В листинге 6 можно увидеть, как, во-первых, в формат **.o** заносятся значения переменных, определяемых в коде макросами группы `MODULE_*()`, а, во-вторых, как формат **.ko** дополняется информацией, необходимой для корректной загрузки модуля: версии ядра, зависимости и т.д. Попробуйте перенести файл любого из созданных модулей на другой компьютер и посмотреть, что из этого получится.

Листинг 7. Попытка использовать модуль на некорректном ядре

```
$ uname -r
2.6.35.14-96.fc14.i686.PAE
$ sudo insmod md1.ko
insmod: error inserting 'md1.ko': -1 Invalid module format
$ dmesg | tail -n1
[20168.420293] md1: version magic '2.6.32.9-70.fc12.i686.PAE SMP mod_unload 686 '
should be '2.6.35.14-96.fc14.i686.PAE SMP mod_unload 686 '
$ lsmod | grep md1
$
```

Как видно, в ходе выполнения возникла ошибка. Но было бы ещё хуже, если бы этот модуль, собранный не для данного ядра, удалось загрузить, так как малейшая ошибка в коде модуля ядра, в отличие от процессов пространства пользователя, может привести к немедленной смерти всей операционной системы. Иногда ядро даже не успевает вывести «предсмертное» сообщение Oops, которым обычно диагностируются критические ошибки (аналог «синего экрана смерти» в MS Windows). Вот к чему могут привести ошибки в коде модуля ядра.

Заключение

После подробного анализа результатов, полученных в ходе практического эксперимента, пришло время перейти к изучению теоретических аспектов разработки модулей ядра. И первым из подобных вопросов будет рассмотрен экспорт символов как со стороны модуля, так и самого ядра.

Об авторе

Олег Цилюрик



Олег Иванович Цилюрик, много лет был разработчиком программного обеспечения в крупных центрах разработки: ВНИИ РТ, НПО "Дельта", КБ ПМ. Последние годы работал над проектами в области промышленной автоматики, IP телефонии и коммуникаций. Автор нескольких книг. Преподаватель тренингового отделения международной софтверной компании Global Logic.

© Copyright IBM Corporation 2012
(www.ibm.com/legal/copytrade.shtml)
Торговые марки
(www.ibm.com/developerworks/ru/ibm/trademarks/)